SQL4Stats: Design and Development of an Interactive Data Science Teaching Resource

Charlie Coleman, in collaboration with Sebastian Pollack

2026-08-20

Abstract

Because SQL and the tools built around it developed primarily within industry rather than academia, there are comparatively few centralized resources connecting SQL syntax to the performance characteristics of the underlying engine, even as journals such as VLDB continue to publish research dissecting those mechanisms at a technical level. Many students who study mathematics or statistics in college go on to positions where they use SQL extensively, but undergraduate math curricula rarely address it directly. Working with Professor Jo Hardin in the Pomona College Statistics Department, we sought to offer a perspective shaped by a small liberal arts institution, where there is a growing student interest in the gap between a traditional statistics education and the data science skills that industry expects. Our goal was to lower the barrier to entry for statisticians transitioning into data science, whether undergraduates learning the fundamentals of SQL or instructors developing a curriculum around it. We addressed this goal by building an interactive website that covers the theoretical foundations of SQL, illustrative simulations of query performance, and tutorials for aspiring SQL practitioners. The project was designed primarily for undergraduate students and instructors with a statistics background, though the content is intended to be accessible to a broad audience.

Motivation

This project began with a straightforward observation: SQL is one of the most widely used languages in data-adjacent industry positions, and yet most undergraduate statistics students never formally encounter it. Statistics departments, understandably, concentrate their curricula on R, inferential theory, and modeling, since these constitute the core of statistical practice as it is taught in an academic setting. A student's first position after college, however, particularly one in data science or data engineering, is far more likely to involve querying a relational

database than fitting a linear model in R. This creates a gap between what students are trained to do and what they are actually asked to do once they begin working.

Part of the reason this gap persists is that SQL does not occupy a natural place within academic curricula. It was developed within, and primarily for, industry–companies like Oracle and Microsoft have even gone so far as to create their own proprietary dialects of the language. Most material written about it outside of the formal database systems literature, consists either of terse reference documentation or fragmented tutorials that treat SQL syntax and performance as separate topics. Comparatively little material has been written specifically to bridge a statistics background into SQL, and even less attempts to explain why a query performs as it does rather than simply which syntax produces a given result.

Our website was designed to address that gap and is organized around a single guiding question: which factors drive SQL query performance? Answering that question requires treating SQL as two layers at once: the logical layer of what is written and the physical layer of what the database engine does to execute it. Nearly every section of the site returns to this same idea in a different context: keys and indexes, join algorithms, file formats, and query optimization are, ultimately, all questions about the distance between a written query and the work the machine must actually perform.

Site Architecture Overview

The site is organized into three main sections: Theoretical Foundations, Theory in Practice, and Tutorials, along with an About page and a References page documenting our sources.

Theoretical Foundations covers three topics, each built as a long-form scrollytelling page: SQL keys, indexes, and B+ trees; the logical and physical mechanics of joins; and file formats, focusing on the comparison between CSV, JSON, and Parquet, along with an introduction to DuckDB.

Theory in Practice contains three fully reproducible simulation-based experiments that put the concepts from the theoretical foundations pages to an empirical test: a comparison of native DuckDB versus direct Parquet querying, a controlled exploration of how Parquet query time scales with file count and row count, and a demonstration of how join algorithm and join order affect PostgreSQL query performance.

Tutorials contains two walkthroughs aimed at getting a reader hands-on with real data: a guide to uploading a messy collection of CSVs into a SQL database and keying it properly, and a comparison of two different meanings of "pivoting" data, pure reshaping versus cross-tabulation, across R, DuckDB, and PostgreSQL.

Theoretical Foundations

SQL, Keys, Indexes, and B+ Trees

With accessibility as a guiding principle, we built the theoretical foundation pages as a single continuous page through which a reader progresses by scrolling rather than by actively navigating between separate views. Because our target audience consists largely of undergraduate students, we considered this an appropriate format for reaching them, given that most readers, our target audience included, are already accustomed to consuming information in this continuous manner.

The first theoretical foundations page introduces the logic of SQL at a high level, beginning with the language itself, some of its dialects, and the structure of the data it is designed to manipulate. We chose to emphasize the relational aspect of SQL from the outset, as this represents SQL's principal advantage over a language such as R or Python, where a practitioner must load and manage a collection of separate CSV files without any mechanism enforcing the relationships among them. To illustrate this concretely, the page explains how keys function: a primary key uniquely identifies each row in a table, and a foreign key is a column that references another table's primary key, thereby establishing and enforcing a relationship between the two. The page then provides the CREATE TABLE and ALTER TABLE syntax required to define these keys, allowing a reader to begin working with relational databases without first needing to understand the structures that underlie them.

The page then proceeds to a deeper layer, addressing indexes and B+ trees. This is where the logical and physical layers of SQL are first explicitly distinguished on the site: a key is a logical constraint on a column, whereas an index is a separate physical data structure that the database maintains to accelerate lookups on that column. We sought to draw a direct connection between the structure of an index and its effect on the efficiency of a specific operation (a value search) since this connection is central to the purpose of introducing indexes in the first place. The page explains why a B+ tree remains shallow, illustrating the concept of fanout with visual aids. It concludes with a discussion of when an index is actually beneficial: indexes are designed for transactional row lookups rather than the full-column scans that analytical workloads require, a distinction that establishes the foundation for the file formats page later in the section.

Joins

Section 1.2 reinforces the same logical-versus-physical distinction using joins as a second, more concrete example. Because the previous page had just explained the structures underlying keys, joins follow naturally, as keys are the mechanism on which joins are built. The page distinguishes logical joins, inner, left, right, full outer, and cross, which describe what rows a query returns, from physical joins, nested loop, merge, and hash, which describe how the query optimizer actually produces those rows. Throughout the page, animations serve as a

visual aid so that a reader is not merely told the inputs and outputs of a join, but can observe a simplified version of each algorithm in operation and develop an intuition for why it behaves as it does.

Animating each physical join algorithm side by side also provides a reader with a comparative view: why a nested loop join can grow to $O(m \cdot n)$ on unsorted, unindexed tables, why a merge join's cost is instead proportional to the sum rather than the product of the two table sizes, and why a hash join's build-and-probe structure gives it linear time on large, unsorted inputs. Because SQL is a language designed for, and used predominantly in, industry, it was important to emphasize how a practitioner can improve the efficiency of their queries, rather than simply how to write syntactically correct SQL. A concrete understanding of why one join algorithm outperforms another builds an intuition that carries over into stronger problem-solving once a reader begins writing queries against real data.

File Formats, Parquet, and DuckDB

The final subsection describes several file storage formats, from row-based formats such as CSV and JSON to the columnar and hybrid Parquet format, comparing how each performs across different kinds of tasks. Like the first two pages, this one is organized around the same underlying idea: understanding the physical work a computer must perform to answer a query. The page explains why a row-based format retrieves a full record quickly but scans a single column across millions of rows slowly, why a purely columnar format inverts that trade-off, and why Parquet's hybrid row-group structure captures much of the benefit of both. It also explains why Parquet compresses substantially better than CSV or JSON, through columnar grouping and dictionary encoding, illustrated with an empirical comparison across the three formats using a subset of the Stanford Open Policing Project data (Pierson et al. 2023), the same dataset used later in the Theory in Practice simulations.

This page also introduces DuckDB, an embedded, in-process SQL engine that requires no server setup and can query Parquet files directly from disk using its read_parquet() function, exploiting the statistics stored in a Parquet file's footer to skip row groups that cannot match a query's filter. We included a local alternative to a remote SQL server on the site both because it removes a significant setup barrier for a reader attempting to follow along, and because DuckDB's architecture, in-process, columnar, vectorized, represents a genuinely different approach to database design compared with the client-server, row-oriented model that MySQL and PostgreSQL use elsewhere on the site. By approaching efficiency through the lens of file storage, compression, and disk space, this page completes a picture of SQL performance that extends beyond the number of operations a query requires to encompass how the underlying data is physically arranged on disk.

Theory in Practice

Section 2, Theory in Practice, contains three simulation-based experiments that subject the ideas from Section 1 to empirical testing. Every trial is fully reproducible, with the source code included directly within the page alongside a plain-language description of the experiment's methodology and purpose. Presenting these experiments in accessible terms is a deliberate choice, consistent with our broader goal of lowering the barrier to entry into data science for a statistics-trained reader.

Native DuckDB vs. Direct Parquet Querying

Section 2.1 compares query performance on the same dataset stored in two ways: as a native DuckDB database, and as a set of Parquet files queried directly with read_parquet(). The data comes from a subset of three California cities, Los Angeles, San Francisco, and San Jose, in the Stanford Open Policing Project (Pierson et al. 2023). The only difference between the two conditions is the storage format itself; DuckDB executes the query in both cases. Direct Parquet querying requires DuckDB to open, parse, decompress, and query the Parquet files from disk on every execution, with nothing pre-loaded into a database.

Six query types are benchmarked, ranging from a simple COUNT(*) to a cross-table aggregation across all three cities. Each timing result reflects the mean of 20 timed runs, preceded by three discarded warm-up runs, with a fresh DuckDB connection opened and closed for every trial to avoid connection-level caching effects. Differences between the two approaches are tested using a two-sided Wilcoxon rank-sum test at $= 0.05$, because query times are non-negative, right-skewed, and not guaranteed to follow a normal distribution.

The results show a statistically significant difference for every query type, favoring native DuckDB, which is both faster and more consistent across all six queries, most notably on the string-filter query, where direct Parquet querying was more than three times slower. Direct Parquet querying, however, requires approximately one-third of the disk space, roughly 90 MB versus 312 MB for the equivalent native DuckDB database, and can be queried immediately after the files are written, without an ingestion step. The page concludes with practical guidance for selecting a storage format based on a reader's workload: native DuckDB for workflows involving repeated queries over stable data, and direct Parquet when storage efficiency or avoiding ingestion latency outweighs raw query speed.

Parquet Querying Time Exploration

Section 2.2 presents two additional, tightly controlled benchmarking experiments, examining how query time changes as a function of the number of Parquet files queried and, separately, the total number of rows in those files. Both experiments query files directly with read_parquet(), without involving a native database. Two query types serve as benchmarks throughout:

COUNT(*), which requires minimal I/O but must open and process every file, and SELECT c5, which reads a single column across every row and is I/O-intensive but benefits from Parquet's columnar layout. Each timing result reflects the mean of 100 timed runs in the first test or 25 timed runs in the second, preceded by five discarded warm-up runs and one additional discarded cache-primer run.

The first test holds the total row count fixed at one million and varies the number of files across which those rows are split, from a single file up to 200. COUNT(*) time rises steadily and nearly linearly with file count, from 0.67 ms at one file to 9.4 ms at 200 files, even though the total volume of data never changes, because DuckDB must open and parse the footer of every file individually before it can aggregate a result. SELECT c5 changes little at low file counts, since the cost of physically reading and decompressing a column dominates file-opening overhead, but the two trends converge as file count exceeds approximately 50, where coordination overhead again becomes visible. The result indicates that file count matters most for metadata-bound queries such as COUNT(*), while data volume matters most for I/O-bound queries such as SELECT c5.

The second test fixes the file count, comparing a single file against 50 files, and instead varies the total row count from 100 to two million. Here, COUNT(*) exhibits a stable gap between the single-file and 50-file conditions across every row count, confirming that the overhead is tied to file count rather than data volume: the difference does not grow as rows increase, though it diminishes as a proportion of overall query time. SELECT c5 behaves differently, as both conditions grow together as row count increases because the dominant cost is decompressing an increasingly large column, and the gap between them narrows in relative terms as the volume of data read comes to dominate the query.

Together, the two tests support the same practical conclusion: the optimal organization of a set of Parquet files depends on the query pattern that dominates a given workload. Workloads composed of lightweight aggregations and metadata queries benefit from fewer, larger files, while workloads that scan large columns across many rows are less sensitive to file count. Neither a single, very large file nor a great many small ones is universally optimal.

Join Algorithms and Join Order in PostgreSQL

Section 2.3 turns to PostgreSQL and isolates the effect of two decisions a query optimizer normally makes independently: which physical join algorithm to use, nested loop, hash, or merge, and in what order to bring tables into a multi-table join. The simulation uses STATS-CEB, an anonymized dump of the Stats StackExchange (Han et al. 2021), joining four tables, votes, posts, users, and badges, with identical WHERE filters but two different table orderings in the FROM clause: a "good" order that leads with the two most selectively filtered tables, and a "bad" order that leads with the two large, unfiltered tables. Both queries return an identical count.

By setting PostgreSQL session variables (enable_nestloop, enable_hashjoin, enable_mergejoin, and join_collapse_limit), it is possible to force one join algorithm and one join order at a time, a configuration the planner would never adopt voluntarily, in order to observe the effect of each choice in isolation. Crossing three algorithms with two orders yields six timing conditions, each run ten times using a warm-cache methodology, a discarded warm-up run followed by ten timed EXPLAIN ANALYZE runs. The results indicate that a poor join order degrades performance under every algorithm, but affects nested loop joins most severely, since a nested loop probes its inner table once per row of its outer table, causing that probe count to increase substantially when the outer table is large and unfiltered. A hash join is considerably more resilient to a poor order because it builds a hash table on one input and streams the other past it; a merge join falls between the two, as its cost is dominated by sorting rather than by join order itself.

The page also demonstrates how to interpret a query plan, using EXPLAIN ANALYZE output from both the auto-optimized query and the manually forced worst case. Left unconstrained, the planner reorders the bad-order query into something close to the good order on its own, keeping early intermediate results small; when forced into the literal bad order with join_collapse_limit set to 1, the same query's runtime increases from approximately 1,306 ms to approximately 5,050 ms, a nearly fourfold penalty, because a substantially larger number of rows must be carried through every subsequent join. A concluding analysis of estimated versus actual row counts, using a q-error statistic, shows that the planner's row estimates are inaccurate across all three plans, most severely at a badges index scan that misestimates by a factor of roughly a million and a half. What actually distinguishes a fast plan from a slow one, however, is not the magnitude of that error but the number of rows that pass through the node where the error occurs. The page concludes with a practical takeaway: join order matters most for nested loop joins, and because the planner's choices follow directly from its row estimates, maintaining current table statistics and writing selective filters is generally a more effective strategy than attempting to outperform the optimizer manually.

Tutorials

SQL Upload Guide

The first tutorial guides a reader through building and keying a functioning SQL database from scratch, using a dataset of Saturday Night Live cast members, episodes, sketches, and appearances scraped from a public GitHub repository (Hilleckes and Morris 2017). We selected this dataset because it is unfamiliar and engaging, a reader is more likely to remain interested in a walkthrough involving SNL data than in a generic sales or employee dataset, while still exhibiting the messy, real-world problems, orphaned foreign keys, ambiguous natural keys, mismatched column types, that a reader will encounter with any dataset they bring into a database themselves.

The guide is organized into two parts. The first addresses transferring a folder of scraped CSV files into a MariaDB server from R: connecting with credentials stored as environment variables rather than hardcoded values, validating file contents before uploading, converting file names into valid SQL table names, and uploading data in chunks rather than as a single, large INSERT statement. The second, longer part concerns keys specifically, since keys are what actually transform a set of uploaded tables into a relational database rather than simply a collection of tables residing on the same server. It covers inspecting an unfamiliar database with DESCRIBE, verifying whether a candidate column satisfies the three requirements of a valid primary key, no missing values, no duplicates, and a meaningful real-world identifier, identifying orphaned foreign key values with a LEFT JOIN … WHERE parent.key IS NULL pattern, determining how to resolve those orphans, correcting storage engine and data type mismatches that would otherwise prevent a foreign key constraint from being created, and finally creating the primary and foreign keys themselves.

The guide also addresses two situations that arise frequently in practical data modeling: composite keys, in which no single column is unique but a combination of two columns is, and surrogate keys, in which no existing column or combination of columns is guaranteed to be unique, requiring a new auto-incrementing identifier column to be added instead.

Pivot Types: Reshaping vs. Cross-Tabulation

The second tutorial addresses a specific point of confusion for readers coming from R: the term "pivot" encompasses two distinct operations that are easily conflated. Pure reshaping changes the shape of data, from long to wide, without performing any arithmetic; cross-tabulation reshapes and aggregates simultaneously, collapsing multiple rows that share the same row-group and column-group combination into a single cell using a summary function. Using the gapminder dataset, the page demonstrates pure reshaping, country by year, one life expectancy value per cell, through three equivalent approaches: pivot_wider() in R, DuckDB's PIVOT command without a USING clause, and a manual CASE WHEN and GROUP BY query, confirming that all three produce identical output.

It then demonstrates cross-tabulation, continent by year, averaged across countries, through three parallel approaches: a manual CASE WHEN and AVG() query, DuckDB's PIVOT with USING avg(), and PostgreSQL's dedicated crosstab() function, again confirming that the outputs match. The page also addresses a natural follow-up question directly: whether crosstab() can be used for pure reshaping at all. The answer is that it can, technically, though only impractically, since crosstab() presumes that its input has already been aggregated into one row per cell, making it well suited to cross-tabulation but poorly suited to pure reshaping.

Design Rationale

Several choices recur across the site, and we consider them worth discussing directly, as they reflect deliberate pedagogical decisions rather than incidental ones.

Audience and Accessibility

Although sql4stats was designed primarily with undergraduate statistics students and their instructors in mind, we intended the site to be usable by a broad range of readers, regardless of their statistics or programming background. This intention shaped numerous smaller decisions throughout the site: defining terms such as primary key and foreign key in plain language before employing them, presenting worked syntax examples alongside conceptual explanations, and generally favoring an accessible tone over a more formal, textbook-like style.

Scrollytelling Format

The three Theoretical Foundations pages are each constructed in a scrollytelling format, in which a reader progresses through the content by scrolling down a single continuous page rather than navigating between separate pages or tabs. We selected this format deliberately with our target audience in mind: undergraduate students are already accustomed to receiving information in this manner, through the kind of continuous scrolling that characterizes much of the contemporary web, and we sought to present the site's primary instructional content in a format already familiar to readers rather than requiring them to adapt to an unfamiliar format alongside an unfamiliar subject.

Animation and Visualization

The Joins page, in particular, relies on animation as a teaching device, guiding a reader through a simplified, step-by-step version of each physical join algorithm rather than describing it in prose alone. This is significant because the physical layer of SQL, unlike the logical layer, is not visible by default: a reader can observe the syntax they wrote and the rows returned, but not the nested loop, hash table, or sort-merge operation that produced them. Animating this process provides a reader with a visual reference, which we believe builds a more durable intuition than a written description alone, particularly for a comparative question such as why a hash join typically outperforms a nested loop on a large, unsorted table.

Reproducibility as a Teaching Tool

Every experiment in Theory in Practice includes its complete source code directly on the page, alongside a plain-language explanation of the code's function and purpose. We treated reproducibility here as a pedagogical device: a reader may copy the code, execute it against their own data, and observe a comparable result independently, which reinforces the underlying concept more effectively than a results table alone. This approach also models sound methodology, warm-up runs, multiple timed trials, explicit significance testing, for a reader who may later benchmark their own queries.

Conclusion

This website was built to address the question of query performance from as many angles as possible. Across the Theoretical Foundations, Theory in Practice, and Tutorials sections, the site consistently returns to the same underlying distinction, between the SQL a person writes and the physical work a database engine must perform to execute it, whether manifested in the structure of a B+ tree, the algorithm underlying a join, the layout of a file on disk, or the order in which tables are listed in a FROM clause. We hope the site accomplishes its intended purpose: lowering the barrier to entry for a statistics-trained reader who seeks a genuine understanding of SQL, rather than sufficient syntax to function adequately.

Acknowledgments

This project was supported by the Summer Undergraduate Research Program (SURP) at Pomona College and the Kenneth Cooke Research Fellowship. We are grateful to Professor Jo Hardin for her guidance and support throughout this project.

References

Apache Parquet. (2026). Compression — Parquet file format. https://parquet.apache.org/docs/file-format/data-pages/compression/

Han, Y., Wu, Z., Wu, P., et al. (2021). Cardinality estimation in DBMS. Proceedings of the VLDB Endowment, 15(4), 752–765.

Hilleckes, H., & Morris, C. (2017). snldb: A Saturday Night Live database. https://github.com/hh-llcks/snldb

Melnik, S., Gubarev, A., Long, J. J., et al. (2010). Dremel: Interactive analysis of web-scale datasets. Proceedings of the VLDB Endowment, 3. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/36632.pdf

Pierson, E., Corbett-Davies, S., Goel, S., et al. (2023). The Stanford Open Policing Project. https://openpolicing.stanford.edu/

Wickham, H., & Bryan, J. (2023). R packages (2nd ed.). O'Reilly Media. https://r-pkgs.org

Winand, M. (2026). The search tree (B-tree) makes the index fast. https://use-the-index-luke.com/sql/anatomy/the-tree